

Project settings

The *settings.py* file handles the overall settings for the current Django project. Let's take a look at the basic file structure. Here, I will only explain the important sections in this file. It is much easier to comprehend other options since they are self-explanatory, thanks to the excellent commenting:

```
slinux@hackbox:~/LFX/helloweb$ less settings.py:
DATABASE_ENGINE = '' # a 'postgresql_psycopg2', 'postgresql', 'mysql',
# 'sqlite3' or 'oracle'.
DATABASE_NAME = '' # Or path to database file if using sqlite3.
DATABASE_USER = '' # Not used with sqlite3.
DATABASE_PASSWORD = '' # Not used with sqlite3.
DATABASE_HOST = '' # Set to empty string for localhost.
# Not used with sqlite3.
DATABASE_PORT = '' # Set to empty string for default.
# Not used with sqlite3.
```

This section is used to provide the database settings. In Django, we use the models concept to deal with the database and datamodels. It supports all the various database servers that are widely used. To use a MySQL server, we can configure the settings as follows:

```
DATABASE_ENGINE = 'mysql'
DATABASE_NAME = 'database_name'
DATABASE_USER = 'database_username'
DATABASE_PASSWORD = 'password'
```

We may use CSS and JS files as resource files for our Web page. In order to reference them we need to provide the absolute path and the media URL path. For example:

```
MEDIA_ROOT = '/home/slinux/LFX/helloweb/media'
MEDIA_URL = 'http://localhost:8080/helloweb/media'
```

We'll need to specify the *MEDIA_URL* parameter in the URL format that points to the absolute path. We will use the *MEDIA_URL* to access media files like CSS, JS and other files accessible throughout the Web URL.

To facilitate template support, we use template settings in the *settings.py* file. We specify the templates directory path using the *TEMPLATE_DIRS* tuple as follows:

```
TEMPLATE_DIRS = (
    '/home/slinux/LFX/helloweb/templates_dir1',
    '/home/slinux/LFX/helloweb/templates_dir2'
)
```

The *INSTALLED_APPS* section facilitates using third-party Django modules along with our project. For example, if we need a user registration application, we need not write it from scratch. We can download it and attach that application to our project by keeping the application in our project directory and adding a line in the *INSTALLED_APPS* tuple. When we add some applications to the current project, we have to add its reference path to this tuple:

```
INSTALLED_APPS = (
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.sites',
)
```

Views and templates

Writing dynamic pages in Django is very easy. Django puts different concepts into practice—like views and templates.

Views are routines that actually return an output as a Web page, i.e., a dynamic page. We can keep several view routines for different URLs. When a URL is requested by a browser, the Django server checks its mapping from the *urls.py* file, and the corresponding function specified in it is displayed as the output.

Templates in Django are similar to what we mean by templates, in general. The basic idea is to keep the HTML user interface code separate from the logic code and data processing side. Templates provide a higher abstraction over the logic. We can use the variables and data that are returned by the core functions as required by the user interface part. Thus we can simply write the user interface code without bothering about the core logic.

We will now look at how to write view functions in practice. We can place our views in any custom *.py* file. But we have to specify its path in the *urls.py* URL configuration file.

Consider the following code fragment:

```
from django.http import HttpResponseRedirect
def index(request):
    return HttpResponseRedirect('hello World')
```

This is the simplest form of the view function.

To map a URL to a view function, we need to append the following line to it:

```
(r'helloweb/$', 'helloweb.views.index'),
```

Here, *r'helloweb/\$'* is the URL regular expression; the *^* symbol specifies that the page URL starts with *helloweb*, and *\$* specifies that the URL ends with *helloweb*. Hence, the URL is matched to the *index()* function only when a correct 'helloweb' occurs in the URL.

If the regular expression was just *r'helloweb'*, it can match any URL, like *something_helloweb_something*.

The 'request' parameter in a view function is the *HttpRequest* object. We can obtain data related to a session, GET request, and POST request from the 'request' object.

To obtain POST request variables, we can use *post_var = request.POST*. It will return a dictionary that contains all post variables and their values. We can use *get_var = request.GET* to obtain all GET variables and values. Similarly, for a Session, use *session_var = request.session*.

To check whether a variable exists in a dictionary, we can make use of the *has_key()* method. Have a look at the modified index view function that makes use of GET variables: